

A Local Information Aggregation-Based Multiagent Reinforcement Learning for Robot Swarm Dynamic Task Allocation

Yang Lv[✉], Jinlong Lei[✉], *Member, IEEE*, and Peng Yi[✉], *Member, IEEE*

Abstract—In this article, we explore how to optimize task allocation for robot swarms in dynamic environments, emphasizing the necessity of formulating robust, flexible, and scalable strategies for robot cooperation. We introduce a novel framework using a decentralized partially observable Markov decision process (Dec-POMDP), specifically designed for distributed robot swarm networks. At the core of our methodology is the local information aggregation multiagent deep deterministic policy gradient (LIA-MADDPG) algorithm, which merges centralized training with distributed execution. During the centralized training phase, a local information aggregation (LIA) module is meticulously designed to gather critical data from neighboring robots, enhancing decision-making efficiency. In the distributed execution phase, a strategy improvement method is proposed to dynamically adjust task allocation based on changing and partially observable environmental conditions. Our empirical evaluations show that the LIA module can be seamlessly integrated into various centralized training and decentralized execution (CTDE)-based multiagent reinforcement learning (MARL) methods, significantly enhancing their performance. Additionally, by comparing LIA-MADDPG with six conventional reinforcement learning algorithms and a heuristic algorithm, we demonstrate its superior scalability, rapid adaptation to environmental changes, and ability to maintain both stability and convergence speed. These results underscore LIA-MADDPG's outstanding performance and its potential to significantly improve dynamic task allocation in robot swarms through enhanced local collaboration and adaptive strategy execution.

Index Terms—Dynamic task allocation, multirobot systems, networked robots, robot swarm.

I. INTRODUCTION

WITH the continuous advancement of modern technology, robot swarms have emerged as a significant

research area, adept at handling complex tasks such as UAV swarms [1], *ad hoc* network relay [2], and cooperative tracking control [3]. These swarms, comprising numerous small robots, excel in cooperative collaboration, underscoring the potential of collective intelligence [4]. However, effectively coordinating large-scale swarms remains challenging, especially when task requirements evolve in real time. The associated task allocation problem—distributing multiple, often mobile, tasks among robots to optimize performance—has thus gained considerable attention in diverse domains, including industrial automation [5], emergency rescue [6], and environmental monitoring [7].

Although substantial progress has been made in task allocation, existing literature primarily focuses on scenarios with infrequent, discrete changes, such as the sudden addition or removal of tasks or robots [8], [9]. These approaches often rely on periodic adjustments and may fail to address the more demanding setting of continuously changing task locations and highly dynamic task requirements [10]. As swarm sizes increase, task allocation complexity escalates due to the exponentially growing search space [11]. Moreover, both inter-robot communication constraints and time-sensitive reallocation demands present additional obstacles to achieving robust and efficient task assignments in real-world scenarios.

Traditional planning-based methods for task allocation can be broadly categorized into centralized and distributed approaches. Centralized methods rely on a central planner collecting global task information and assigning tasks accordingly [12], [13], [14]. However, real-world tasks often change continuously, making purely one-shot allocations impractical [15]. Consequently, many researchers turn to dynamic task allocation with periodic re-planning [16], [17], though the computational load can become considerable for large robot teams. In contrast, distributed methods usually offer greater scalability and computational efficiency [18], [19], [20]. Prominent techniques include the auction algorithm [21] and the contract network algorithm [22]—along with subsequent variants [23], [24], [25]—which have been widely applied in robot team collaborations. While these approaches work well for moderate-size systems, their efficiency drops for very large robot populations. A game-theoretic framework [26] has also been proposed to converge toward Nash equilibria in polynomial time under stable communication topologies. However, in strongly dynamic scenarios such as emergency rescue [27], robot states and communication links evolve rapidly, making it challenging for traditional distributed algorithms to maintain robust, real-time task allocations.

Received 13 November 2023; revised 28 May 2024, 11 September 2024, 8 January 2025, and 13 February 2025; accepted 30 March 2025. Date of publication 23 April 2025; date of current version 4 June 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFA1004701, in part by the National Natural Science Foundation of China under Grant 72271187 and Grant 62373283, and in part by Shanghai Municipal Science and Technology Major Project under Grant 2021SHZDZX0100. (Corresponding author: Jinlong Lei.)

Yang Lv is with Shanghai Research Institute for Intelligent Autonomous Systems, Tongji University, Shanghai 200092, China (e-mail: 726564418@qq.com).

Jinlong Lei and Peng Yi are with the Department of Control Science and Engineering, Tongji University, Shanghai 201804, China, also with Shanghai Research Institute for Intelligent Autonomous Systems, Shanghai 201804, China, also with Shanghai Institute of Intelligent Science and Technology, Tongji University, Shanghai 200092, China, also with the National Key Laboratory of Autonomous Intelligent Unmanned Systems, Shanghai 201210, China, and also with the Frontiers Science Center for Intelligent Autonomous Systems, Ministry of Education, Shanghai 201210, China (e-mail: leijinlong@tongji.edu.cn; pengyi@tongji.edu.cn).

Digital Object Identifier 10.1109/TNNLS.2025.3558282

Recently, multiagent reinforcement learning (MARL) methods built upon the centralized training and decentralized execution (CTDE) paradigm have shown considerable promise for addressing dynamic task allocation in complex scenarios [28], [29], [30]. Originating with [31], a foundational tabular-based multiagent Q -learning framework was developed to manage dynamic tasks in unpredictable environments. Continuing this trend, approaches such as multiagent actor-critic (MAAC) [32] and asynchronous multiagent deep deterministic policy gradient (MADDPG) [33] improved training efficiency and convergence in highly dynamic conditions. Concurrent research integrated MAPPO [34] and QMIX [35] to handle distinct challenges like multiobjective coordination and resource contention. Despite these advances, most studies concentrate on discrete events or partially dynamic settings, without fully addressing situations where task locations change continuously. Moreover, existing CTDE frameworks often face severe scalability issues—the well-known dimensionality explosion problem—when the number of agents grows.

To address these challenges, we propose local information aggregation MADDPG (LIA-MADDPG), a distributed MARL framework based on local information aggregation (LIA). By focusing on key neighboring information, this approach alleviates the dimensionality explosion problem and integrates global training with local execution under the CTDE paradigm, enhancing both the stability of policy learning and real-time adaptability. The main contributions of this article are as follows.

- 1) We develop a decentralized partially observable Markov decision process (Dec-POMDP)-based framework tailored to large-scale robot swarms in dynamic environments, enabling cooperative decision-making under partial observability and limited communication.
- 2) We introduce the LIA module to tackle dimensionality explosion in MARL. By focusing on locally relevant neighbors, each robot avoids unnecessary global information, reducing computational overhead and improving decision quality. A Taylor-based theoretical analysis shows that LIA mitigates noise in the global Q -function, offering a solid theoretical foundation for the module's efficacy.
- 3) Building on the CTDE paradigm, we propose the LIA-MADDPG framework, which incorporates the LIA module for efficient policy learning. In the distributed execution phase, an optimization step ensures continuous policy refinement in changing environments, allowing robots to adapt their allocation strategies in real time based on limited yet informative local observations.
- 4) We benchmark LIA-MADDPG against six state-of-the-art MARL algorithms and a heuristic baseline. Results demonstrate superior performance in convergence speed, stability, and scalability—particularly for larger robot groups. Moreover, we integrate the LIA module with alternative MARL methods (e.g., MAAC and MAPPO) to validate its broad applicability. Ablation studies confirm that local relevance and aggregation-weight mechanisms are pivotal for overall performance gains.

The article is organized as follows. Section II offers a comprehensive investigation of the robot swarm dynamic task allocation problem, along with its reformulation as a Dec-POMDP model. Section III provides an in-depth description of the proposed method LIA-MADDPG. Section IV presents simulation results and the associated discussions. Lastly, some concluding remarks are given in Section V.

II. PROBLEM DESCRIPTIONS

In this section, we model the robot swarm task allocation problem in dynamic task environments and then reformulate it as a Dec-POMDP.

A. Robot Swarm Task Allocation Problem

This problem involves assigning a swarm of N robots to M mobile tasks. Each task may move continuously in the environment, and each robot can dynamically choose which task to pursue based on local observations.

1) *Movable Task Set*: Let $\mathcal{M} = \{1, 2, \dots, M\}$ denote the set of tasks over time indices $\mathcal{T} := \{0, 1, \dots, T\}$. Each task $j \in \mathcal{M}$ has speed v_j^m and angle $\theta_j^{m,t}$ at time t , and its position $P_j^{m,t} = (x_j^{m,t}, y_j^{m,t})$ evolves as

$$P_j^{m,t+1} = \begin{cases} x_j^{m,t} + v_j^m \tau \cos(\theta_j^{m,t}) \\ y_j^{m,t} + v_j^m \tau \sin(\theta_j^{m,t}) \end{cases} \quad (1)$$

where τ represents the decision time step. Each task has a capacity $\bar{h}_j$, indicating the maximum number of robots it can accommodate.

2) *Robot Swarm Network*: The robot swarm is represented by a network $\mathcal{G} = \{\mathcal{N}, \mathcal{E}\}$, where $\mathcal{N} = \{1, 2, \dots, N\}$ are the robots, and $\mathcal{E} \subset \mathcal{N} \times \mathcal{N}$ denotes perception links. Each robot i can observe task states and receive limited information from neighbors $\mathcal{N}_i \triangleq \{i' \in \mathcal{N} : (i, i') \in \mathcal{E}\}$. We assume that each robot $i \in \mathcal{N}$ is a rational agent, which can instantly access the status of all tasks and information from neighboring robots $i' \in \mathcal{N}_i$. Based on the observed and received information and the current environment, robot $i \in \mathcal{N}$ can dynamically choose target tasks, and subsequently, move at a constant velocity v_i^r toward the chosen task.

Define $g_i^t \in \mathcal{M}$ as the target task of robot $i \in \mathcal{N}$ at time $t \in \mathcal{T}$, and $d_{i,g_i^t}^t \in \mathbb{R}$ as the Euclidean distance between robot i and its target task g_i^t at the t -th decision moment. Let $d_{\text{bind}} > 0$ denote the predefined association distance, and $T_i \in \mathcal{T}$ be the finish time of robot i . If at some time t , robot i 's distance from its target task satisfies $d_{i,g_i^t}^t \leq d_{\text{bind}}$, then robot $i \in \mathcal{N}$ becomes bound to this task $d_{i,g_i^t}^t \in \mathcal{M}$, and $T_i \triangleq t$ is set as the binding time. At time T_i , the robot is considered to have completed the task assignment, and it cannot change its target task anymore. Let $P_i^{r,t+1} = (x_i^{r,t+1}, y_i^{r,t+1})$ represent the coordinates of robot $i \in \mathcal{N}$, and $\lambda_{i,g_i^t}^{r,t+1} = ((y_{g_i^t}^{m,t} - y_i^{r,t}) / (x_{g_i^t}^{m,t} - x_i^{r,t}))$ represent the direction vector of robot $i \in \mathcal{N}$. Then, each robot moves toward the chosen target task and can update their motion state according to (2). It should be emphasized that once the robot

is bound, its motion state will consistently synchronize with its target task

$$P_i^{r,t+1} = \begin{cases} x_i^{r,t} + v_i^r \tau \cos(\lambda_{i,g_i}^{t+1}) \\ y_i^{r,t} + v_i^r \tau \sin(\lambda_{i,g_i}^{t+1}) \end{cases} \quad (2)$$

3) *Utility Function*: Let u_i be the utility for robot $i \in \mathcal{N}$. It comprises a reward u_i^1 and a movement cost reward u_i^2 . Suppose robot i binds to task j at time T_i . We define h_j^t as the number of robots already bound to task j at time t , and $r_{i,j}$ as the reward robot i gains from task j . Then

$$u_i^1 = \begin{cases} r_{i,j}, & \text{if } h_j^{T_i} < \bar{h}_j \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

The second component is the cost incurred by the robot $i \in \mathcal{N}$ while moving toward the target task. It is denoted as $u_i^2 = \sum_{t=1}^{T_i} v_i^r \tau$.

Therefore, for each robot $i \in \mathcal{N}$ with the assigned task denoted as $j = g_i^{T_i}$, its utility function can be described below

$$u_i = u_i^1 - u_i^2 = \begin{cases} r_{i,j} - \sum_{t=1}^{T_i} v_i^r \tau, & \text{if } h_j^{T_i} < \bar{h}_j \\ - \sum_{t=1}^{T_i} v_i^r \tau, & \text{otherwise.} \end{cases} \quad (4)$$

4) *Optimization Problem Formulation*: Our goal is to assign tasks to each robot to maximize the swarm's total utility

$$\max_{\pi_i} \sum_{i \in \mathcal{N}} u_i(\pi_i) \quad (5)$$

where $\pi_i = \{g_i^1, g_i^2, \dots, g_i^T\}$ is the decision variable for robot $i \in \mathcal{N}$, which represents the target task selected by the robot throughout the time series $\mathcal{T}$. We must adhere to the following constraints.

- 1) $T = |\mathcal{T}| = \max_{i \in \mathcal{N}} T_i$, where $d_{i,g_i}^{T_i} \leq d_{\text{bind}}$.
- 2) Once robot $i \in \mathcal{N}$ binds at time T_i , it cannot switch tasks thereafter.
- 3) Each robot can only choose one task at each time t .

Let $x_{i,j}(t) \in \{0, 1\}$ be an indicator of whether robot i chooses task j at time t . The constraints then ensure $\sum_{j \in \mathcal{M}} x_{i,j}(t) = 1 \forall i \in \mathcal{N} \forall t \in \mathcal{T}$.

Formally, the allocation problem is

$$\begin{cases} \max_{\pi_i} \sum_{i \in \mathcal{N}} u_i(\pi_i) \\ \text{s.t. } T = |\mathcal{T}| = \max_{i \in \mathcal{N}} T_i, d_{i,g_i}^{T_i} \leq d_{\text{bind}} \\ g_i^t = g_i^{T_i} \quad \forall t \in [T_i, T] \\ \sum_{j \in \mathcal{M}} x_{i,j}(t) = 1 \quad \forall i \in \mathcal{N} \quad \forall t \in \mathcal{T} \\ x_{i,j}(t) = \begin{cases} 1, & j = g_i^t \\ 0, & j \neq g_i^t \end{cases} \end{cases} \quad (6)$$

B. Reformulation as a Decentralized Partially Observable Markov Decision Process

To handle limited observations in the robot swarm task allocation problem, we model it as a Dec-POMDP [36]. The

environment is defined by the tuple $\mathcal{H} = \{\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{Z}, \mathcal{O}, \mathcal{R}\}$, where each robot $i \in \mathcal{N}$ observes a partial state and acts accordingly.

1) *State Space*: Each robot's state $s_i^{r,t} = \{x_i^{r,t}, y_i^{r,t}, v_i^r, \lambda_i^t, c_i^t\}$ includes position $(x_i^{r,t}, y_i^{r,t})$, velocity v_i^r , heading λ_i^t , and a binary status c_i^t indicating whether the robot is bound to a task. The state information of task $j \in \mathcal{M}$ is defined as $s_j^{m,t} = \{x_j^{m,t}, y_j^{m,t}, v_j^m, \theta_j^{m,t}, h_j^{m,t}\}$. These symbols are consistent with the previous text. Thus, the state space $\mathcal{S}$ consists of two components: the robots' state space $\mathcal{S}^r = \{s_1^{r,t}, s_2^{r,t}, \dots, s_N^{r,t}\}$ and the tasks' state space $\mathcal{S}^m = \{s_1^{m,t}, s_2^{m,t}, \dots, s_M^{m,t}\}$.

2) *Action Space*: At each time step t , robot $i \in \mathcal{N}$ chooses an action a_i^t by selecting a target task g_i^t . This is encoded as an M -dimensional one-hot vector. If $c_i^t = 0$ (already bound), then $a_i^{t+1} = a_i^t$. The joint action is $\mathcal{A} = \{a_1^t, a_2^t, \dots, a_N^t\}$.

3) *State Transition Function*: The environment evolves according to $\mathcal{P}(\mathcal{S}, \mathcal{A})$: $\mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}'$. For tasks, the transition updates positions via (1) and adjusts h_j^{t+1} (the number of bound robots). Therefore, the task state transition function $\mathcal{P}_j^m$ for task $j \in \mathcal{M}$ is as follows:

$$s_j^{m,t+1} = \left\{ x_j^{m,t} + v_j^m \tau \cos(\theta_j^{m,t}), y_j^{m,t} + v_j^m \tau \sin(\theta_j^{m,t}), v_j^m, \theta_j^{m,t+1}, h_j^{t+1} \right\}. \quad (7)$$

For each robot, transitions depend on whether it remains unbound ($c_i^{t+1} = 1$) and continues moving (2) or becomes bound ($c_i^{t+1} = 0$), synchronizing with the task's motion. Hence, the transition function $\mathcal{P}_i^r$ for robot $i \in \mathcal{N}$ after executing action a_i^t can be expressed as follows:

$$s_i^{r,t+1} = \begin{cases} \{x_{g_i^t}^{m,t+1}, y_{g_i^t}^{m,t+1}, v_{g_i^t}^m, \theta_{g_i^t}^{m,t+1}, 0\}, & \text{if } c_i^t = 0 \\ \{x_i^{r,t+1}, y_i^{r,t+1}, v_i^r, \lambda_i^{t+1}, 1\}, & \text{if } c_i^t = 1, c_i^{t+1} = 1 \\ \{x_i^{r,t+1}, y_i^{r,t+1}, v_i^r, \lambda_i^{t+1}, 0\}, & \text{if } c_i^t = 1, c_i^{t+1} = 0. \end{cases} \quad (8)$$

4) *Observation Space*: Each robot has limited perception and communication capabilities. Let $\mathcal{N}_i^t = \{na_1^t, na_2^t, \dots, na_{\alpha_i}^t\}$ be the set of its neighboring robots at time $t \in \mathcal{T}$, where α_i is determined by robot i 's perception range. Consequently, each robot observes only a portion of the global state, decomposed into three components.

- 1) *Self-state*: $o_{\text{Self}}^t = s_i^{r,t}$, describing the robot's own position, velocity, and binding status.
- 2) *Task-relative information*: $o_{\text{Task}}^t = \{\Delta x_{i,j}^t, \Delta y_{i,j}^t, \Delta v_{i,j}^t, \Delta \theta_{i,j}^t, h_j^t, \kappa_{i,j} \mid \forall j \in \mathcal{M}\}$. These features capture robot-task relative positions, velocities, angles, the number of robots already bound to each task, and a normalized factor $\kappa_{i,j}$ indicating how favorable that task is for robot i .
- 3) *Neighbor-relative information*: $o_{\text{Neighbor}}^t = \{\Delta x_{i,i'}^t, \Delta y_{i,i'}^t, \Delta v_{i,i'}^t, \Delta \theta_{i,i'}^t, g_{i'}^{t-1} \mid \forall i' \in \mathcal{N}_i^t\}$. This records relative states and recent actions of neighboring robots.

Hence, each robot's local observation is $o_i^t = \{o_{\text{Self}}^t, o_{\text{Task}}^t, o_{\text{Neighbor}}^t\}$.

5) *Reward Function*: Designing an informative reward is key to guiding robots toward good allocation strategies, especially under sparse rewards. We, therefore, create a three-component reward

$$r_t^i = r_t^{i,\text{dis}} + r_t^{i,\text{step}} + r_t^{i,\text{final}} \quad (9)$$

where the first two items are designed to be non-sparse, and the last term represents the ultimate reward obtained when a robot reaches a target task.

In detail, $r_t^{i,\text{dis}} = -\varphi_2$ is a constant negative reward given to robot $i \in \mathcal{N}$ at each time step until reaching its target task. $r_t^{i,\text{step}}$ is designed based on the utility function u_i^l and aims at guiding robots to satisfy the limited demand constraints of tasks. It is defined as

$$r_t^{i,\text{step}} = \varphi_3 (\bar{h}_j - h_j^t). \quad (10)$$

The final term $r_t^{i,\text{final}}$ represents the ultimate reward obtained when a robot reaches a target task and is defined by

$$r_t^{i,\text{final}} = \begin{cases} \varphi_1 r_{i,j}, & \text{if } h_j^t < \bar{h}_j \\ 0, & \text{otherwise.} \end{cases} \quad (11)$$

Additionally, it is necessary to take a balance between φ_1 and φ_2 to ensure that robots are still influenced by the penalty term $r_t^{i,\text{dis}}$ when frequent target switching occurs.

III. NOVEL MARL ALGORITHM WITH LIA

In this section, we systematically introduce our novel MARL algorithm with LIA, including the main components of the algorithm and the design inspiration behind them.

A. Key Modules and Mechanisms of LIA-MADDPG

MADDPG is a well-known multiagent deep RL algorithm for mixed cooperative-competitive tasks [37], but it struggles to scale to large robot swarms. To address this, we propose LIA-MADDPG, which adds an LIALIA module and leverages policy sharing.

1) *Shared Policy Network*: In standard MADDPG, each of the N agents keeps its own policy μ_i and value Q_i networks (plus target networks). However, for large homogeneous robot swarms, we adopt a shared policy $\bar{\mu}$ (parameters θ^a) and value $\bar{Q}$ with parameters θ^q (parameters θ^a), each with corresponding target networks $\bar{\mu}'(\theta^a)$ and $\bar{Q}'(\theta^q)$. This drastically lowers network complexity and training overhead. Specifically, each robot i uses

$$a_i^t = \bar{\mu}(o_i^t; \theta^a). \quad (12)$$

Similarly, the action output based on the target network can be defined as

$$a_i^{t+1} = \bar{\mu}'(o_i^{t+1}; \theta^a) \quad (13)$$

where fully connected layers, residual connections [38], and batch normalization [39] enhance training stability and efficiency.

2) *Local Information Aggregation*: The traditional MADDPG framework estimates $Q(o, a)$ by taking the full set of joint observations and actions, which becomes problematic at large scales: the dimensionality grows exponentially with the number of agents, causing serious computational overhead. To handle this, we introduce an LIA module that focuses on locally relevant robots and ensures stable input dimensionality—especially beneficial in large-scale swarm settings.

Concretely, we regard locally related robots as those that are both spatially proximate and executing the same action. Such robots exert a stronger mutual influence. For instance, multiple robots moving together toward the same target may either coordinate or compete for that limited-resource task, amplifying the local impact of their decisions. Formally, for each robot i , we define a set $\mathcal{G}_i^t$ as

$$\mathcal{G}_i^t = \underbrace{\{na_1^t, na_2^t, \dots, na_{a_i}^t\}}_{\text{nearby robots}} \cup \underbrace{\{i' \in \mathcal{N} | a_{i'}^t = a_i^t\}}_{\text{action synchronization}} \quad (14)$$

where $\mathcal{N}_i^t = \{na_1^t, na_2^t, \dots, na_{a_i}^t\}$ denotes the set of neighboring robots within a certain distance, and $\mathcal{L}_i^t \triangleq \{i' \in \mathcal{N} : a_{i'}^t = a_i^t\}$ includes those sharing i 's current action. By capturing both spatial and action-related cues, we effectively model local group dynamics while limiting the dimensional explosion of global information. This enriched local context drives more precise credit assignment and better coordination in large-scale multirobot tasks.

Local information set: Define the local information for robot i as the collection of observations and actions from its locally related robots, represented as $L_i = (o_k^t, a_k^t) | k \in \mathcal{G}_i^t$, where o_k^t and a_k^t represent the observation vector and the action vector of robot k at time t .

To validate the advantages of focusing on locally related robots, we follow the methodology from [40], applying a Taylor expansion of the Q -function around the mean observation and action vectors in (15). By examining the first- and second-order terms, one sees that interactions among distant robots—who rarely affect each other's outcomes—add noise and inflate computation. Conversely, restricting the analysis to a local subset $\mathcal{G}_i^t$ filters out irrelevant deviations and yields more stable gradient estimates. This localized viewpoint lowers both the input dimensionality ($O(|\mathcal{G}_i^t| \cdot (d_o + d_a))$) rather than $O(N \cdot (d_o + d_a))$ and the computational cost of handling second-order interactions (from $O(N^2)$ down to $O(|\mathcal{G}_i^t|^2)$). As a result, agents converge more efficiently to robust local cooperation strategies in large-scale, partially observable environments

$$\begin{aligned} Q(o, a) \approx & Q(\bar{o}, \bar{a}) + \sum_{i=1}^N \left(\frac{\partial Q}{\partial o_i} \delta o_i + \frac{\partial Q}{\partial a_i} \delta a_i \right) \\ & + \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \left(\frac{\partial^2 Q}{\partial o_i \partial o_j} \delta o_i \delta o_j + \frac{\partial^2 Q}{\partial a_i \partial a_j} \delta a_i \delta a_j \right. \\ & \left. + \frac{\partial^2 Q}{\partial o_i \partial a_j} \delta o_i \delta a_j \right) + \mathcal{H}_3 \end{aligned} \quad (15)$$

where $\delta o_i = o_i - \bar{o}_i$ represents the deviation of the observation from its mean, $\delta a_i = a_i - \bar{a}_i$ represents the deviation of the action from its mean, and $\mathcal{H}_3$ denotes higher-order terms

(third-order and above). A detailed derivation of the Taylor expansion is provided in the supplementary material.

However, the dimensionality of the local information vector for agent i depends on the size of $\mathcal{G}_i^t$, which may vary between agents and change dynamically over time. This variability can cause inconsistencies in the input vector dimension, affecting network training and convergence. To address this, we draw inspiration from the mean-field reinforcement learning approach proposed in [40] and design an LIA function, φ_i . The function φ_i aggregates the local observations and actions of agents in $\mathcal{G}_i^t$ into a fixed-dimensional vector suitable for input into the value network. The LIA function φ_i is defined as follows:

$$\varphi_i(o_k^t) = \sum_{k \in \mathcal{G}_i^t} w_{i,k}^t o_k^t \quad (16)$$

$$\varphi_i(a_k^t) = \sum_{k \in \mathcal{G}_i^t} w_{i,k}^t a_k^t \quad (17)$$

where $w_{i,k}^t$ is distance-dependent weight coefficients defined based on the distance $d_{i,k}^t$ between robot $i \in \mathcal{N}$ and its locally relevant robot $k \in \mathcal{G}_i^t$. These weight coefficients are assigned to each locally relevant robot $k \in \mathcal{G}_i^t$

$$w_{i,k}^t = \frac{\exp(\beta \ln(d_{i,k}^t))}{\sum_{k \in \mathcal{G}_i^t} \exp(\beta \ln(d_{i,k}^t))}. \quad (18)$$

Remark: The weighting method in (18) and the attention mechanism [41] are two distinct approaches for aggregating agent information in multiagent systems. In (18), weights $w_{i,k}^t$ are determined by physical distance $d_{i,k}^t$ via an exponential function scaled by β , making it ideal for scenarios like robotic swarms where spatial relationships are key. Attention mechanisms, however, adaptively assign weights from data, capturing complex interactions beyond proximity, making them suitable for tasks requiring nuanced context.

The aggregation function φ_i integrates observations and actions from nearby robots, ensuring consistent input dimensions for the value network while capturing local dynamics. By reflecting both environmental conditions and coordinated actions, φ_i enhances decision-making and improves group efficiency in multiagent systems.

3) *Shared Value Network:* We introduce a novel centralized value function, denoted as G . The corresponding shared value network $\tilde{G}$, takes its own local observation o_i^t and action a_i^t , as well as the aggregated information $\varphi_i(o_k^t)$ and $\varphi_i(a_k^t)$ as input. $\tilde{G}$ then computes the evaluation value q_i^t of robot i at time t

$$q_i^t = \tilde{G}[o_i^t, a_i^t, \varphi_i(o_k^t), \varphi_i(a_k^t); \theta^g], \quad k \in \mathcal{G}_i^t, \quad k \neq i. \quad (19)$$

Likewise, the target shared value network $\tilde{G}'$ follows the same structure:

$$q_i^{t+1} = \tilde{G}'[o_i^{t+1}, a_i^{t+1}, \varphi_i(o_k^{t+1}), \varphi_i(a_k^{t+1}); \theta'^g] \quad k \in \mathcal{G}_i, \quad k \neq i. \quad (20)$$

4) *Network Updating:* In MARL, a critical challenge during the network update process is the environmental non-stationarity arising from the policy changes of other agents during training. Take Ind-DDPG [42] as an example. It learns independently for each agent by treating other agents as

components of the environment and evaluates the Q function according to the following:

$$Q^*(s^t, a_i^t)$$

$$= \sum_{s^{t+1}} p(s^{t+1}|s^t, a_i^t) \left[r_i^{t+1} + \gamma \max_{a_i^{t+1}} Q_i^*(s^{t+1}, a_i^{t+1}) \right]$$

where $p(s^{t+1}|s^t, a_i^t)$ represents the probability of agent i transitioning from state s^t to the next state s^{t+1} when taking action a_i^t . However, it is worth noting that during the learning process, changes in other agents' strategies can introduce non-stationarity in the state transition probability $p(s^{t+1}|s^t, a_i^t)$. Consequently, relying solely on a temporal-difference (TD)-based approach for iteratively learning the Q function may result in suboptimal learning outcomes.

In our study, this problem can be effectively addressed by introducing the extended Q function G , which evaluates the expected total reward that robot $i \in \mathcal{N}$ can obtain based on its own local observation o_i , action a_i and the aggregated information from locally connected robots. We then derive the Bellman optimality equation for the extended Q function of robot i 's policy

$$\begin{aligned} G^*(o_i^t, a_i^t, \varphi_i(o_k^t), \varphi_i(a_k^t)) \\ = \sum_{o^{t+1}} p(o^{t+1}|o^t, a_i^t) \\ \times \left[r_i^{t+1} + \gamma \max_{a_i^{t+1}} G(o_i^{t+1}, a_i^{t+1}, \varphi_i^{t+1}(o_k), \varphi_i^{t+1}(a_k)) \right] \end{aligned} \quad (21)$$

The state transition $p(o^{t+1}|o^t, a_i^t)$ can be decomposed as

$$p(o^{t+1}|o^t, a_i^t) = \sum_{a_{-i}^t} p(a_{-i}^t|o_{-i}^t) p(o^{t+1}|o_i^t, a_i^t, a_{-i}^t) \quad (22)$$

where $p(a_{-i}^t|o_{-i}^t) = \prod_{j \neq i} p(a_j^t|o_j^t)$. For robot $i \in \mathcal{N}$, the source of environmental instability arises from the strategy distributions $p(a_{-i}^t|o_{-i}^t)$ of other robots related to i . In this article, these relevant robots can be approximated as the local relevant robot set $\mathcal{G}_i^t$ of robot i . Therefore, the state transition in (22) can be redefined as

$$p(o^{t+1}|o_i^t, a_i^t) = \sum_{a_k^t} p(a_k^t|o_k^t) p(o^{t+1}|o_i^t, a_i^t, a_k^t)_{k \in \mathcal{G}_i^t, k \neq i} \quad (23)$$

where a_k^t and o_k^t represent the local agent actions and observations incorporated into the extended Q function through the aggregation function φ_i . When we know the actions taken by the relevant robots of robot i , even if the policies change, the environment can be considered static. This is because for any policies $\pi_i \neq \pi'_i, \pi_k \neq \pi'_k$, we have $p(o^{t+1}|o^t, a_i^t, a_k^t, \pi_i, \pi_k)_{k \in \mathcal{G}_i^t, k \neq i} = p(o^{t+1}|o^t, a_i^t, a_k^t) = p(o^{t+1}|o^t, a_i^t, a_k^t, \pi'_i, \pi'_k)_{k \in \mathcal{G}_i^t, k \neq i}$. As a result, we can learn the extended Q function G based on the above equation in a stable environment.

Therefore, the shared value network $\tilde{G}$ can be trained to approximate the extended Q function G by minimizing the squared TD error. The loss function is given as follows:

$$\begin{cases} L(\theta^g) = \mathbb{E}[(y - q_i^t)^2] \\ y = r_i^t + \gamma q_i^{t+1} \end{cases} \quad (24)$$

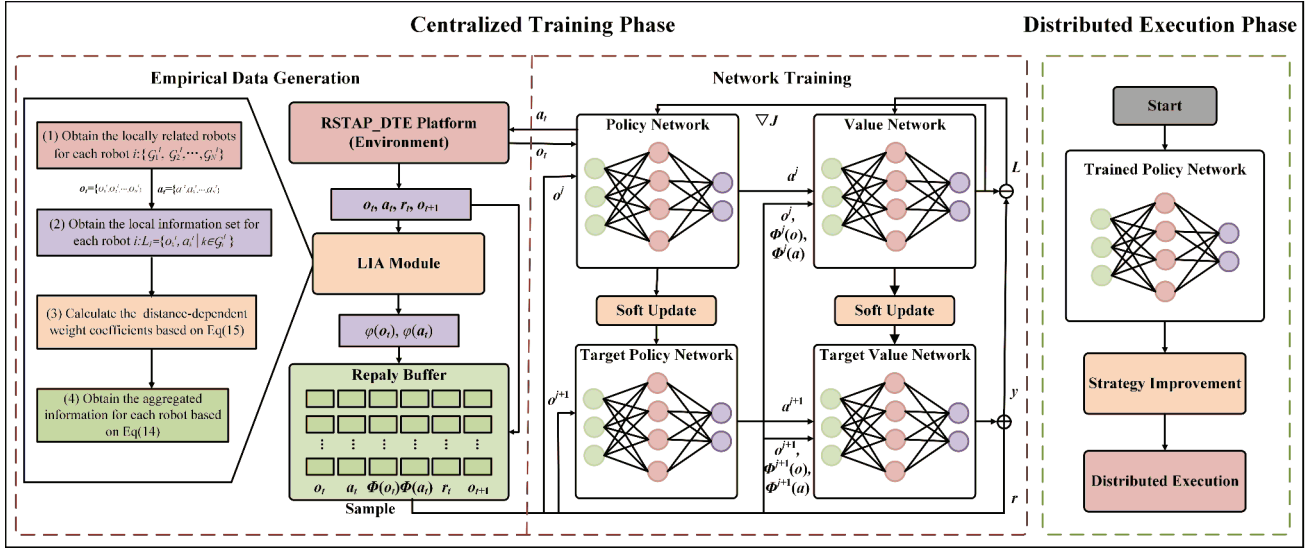


Fig. 1. Structure of the proposed LIA-MADDPG, which consists of two parts: the centralized training and the distributed execution. The LIA-MADDPG employs four types of neural networks: 1) an actor network for generating the action policy; 2) a critic network for evaluating the action policy; 3) a target actor network; and 4) a target critic network for stabilizing the learning process. Furthermore, an LIA module is employed to accelerate the learning process.

where r_i^t is the reward obtained by the robot i at time t , and γ is the discount factor.

The shared policy network $\bar{\mu}$ of robot $i \in \mathcal{N}$ is updated through gradient ascent, aiming to maximize the agent's expected return. The gradient update is performed according to the following equation:

$$\nabla_{\theta^a} J(\theta^a) = \mathbb{E} [\nabla_{\theta^a} \bar{\mu}(a_i | o_i) \nabla_{a_i} q_i^t]. \quad (25)$$

It is crucial to note that, due to the utilization of a shared network approach, in the update process, each robot is required to update the network parameters sequentially based on the sampled data. Sections III-B will present a comprehensive explanation of the specific update process.

B. Overview of the LIA MADDPG Framework

Our LIA-MADDPG framework consists of two main phases: centralized training and distributed execution. It employs four neural networks (actor-critic plus their target versions) and incorporates an LIA module to handle large-scale coordination. The overall structure of the LIA-MADDPG algorithm is illustrated in Fig. 1, and the pseudocode is summarized in Algorithm 1.

1) *Centralized Training Phase*: The centralized training process can be divided into the generation of empirical data and network training, which are executed alternately.

a) *Empirical Data Generation*: As depicted in the leftmost part of Fig. 1, the empirical data generation phase involves each robot continuously interacting with the environment to collect relevant experiential data. During this process, each robot determines its actions a_i based on its current local observation o_i using a shared policy network. Moreover, each robot identifies its set of related robots $\mathcal{G}_i^t$ and its local information set L_i according to the previously described method. The LIA module then aggregates this information, applying distance-dependent weight coefficients to emphasize the influence of closer neighbors. This aggregation process yields a

comprehensive dataset that encapsulates both individual and collective behavior within the environment. The collected data $(o, a, \phi(o), \phi(a), r, o')$ are subsequently stored in the experience replay buffer $\mathcal{D}$, which serves as a critical resource for optimizing the policy and value networks during the network training phase. This process of generating empirical data can be found in lines 6–12 of Algorithm 1.

b) *Network Training*: After the experience data is generated, the data set is extracted from the buffer $\mathcal{D}$ for training based on priority experience replay [43]. At this stage, we need to update $\bar{G}$ and $\bar{\mu}$ according to (24) and (25). First, use the target $\bar{G}'$ to calculate the extended Q function q^{t+1} of the next state, and use the temporal difference method to optimize the parameters of the $\bar{G}$ network based on the value of the extended Q function. Then, use $\bar{G}$ to calculate the extended Q value q^t of the current state-action, and use the calculated Q value and the action a^{t+1} output by $\bar{\mu}$ to update the parameters of the $\bar{\mu}$ network (lines 13–17 of Algorithm 1).

2) *Distributed Execution Phase*: LIA-MADDPG's strength lies in its ability to autonomously develop strategies for problem-solving through extensive instance sampling and learning. This end-to-end approach enables it to quickly produce suitable task allocation solutions without complex heuristic rules. However, it struggles in dynamic environments with partial observability, as robots must adapt to variable conditions and make decisions based on incomplete information. Hence, during the distributed execution phase, strategy optimization methods are required to enhance each robot's decisions based on the shared policy network $\bar{\mu}$. Therefore, there are two process during the distributed execution phase: policy output (lines 2–4 of Algorithm 2) and policy improvement (Lines 5–8 of Algorithm 2), which alternate to facilitate decision-making by each robot.

In the policy output stage, robot feeds the current observation o_i^t into a pretrained neural network $\bar{\mu}(\cdot | \theta^a)$ to obtain the corresponding action a_i^t . This stage maps observations of

Algorithm 1 Training Process of LIA-MADDPG

Input : Max_episode_length T_e , batch size δ , discount factor γ , soft target update rate η

Output: Trained actor network $\bar{\mu}^*$

- 1 **Initialize**: Expand Q network $\bar{G}$ and policy network $\bar{\mu}$ with random weights θ^q, θ^a , target networks $\bar{G}'$ and $\bar{\mu}'$ with weights $\theta'^q \leftarrow \theta^q, \theta'^a \leftarrow \theta^a$, and replay buffer $\mathcal{D}$
- 2 **while** Training is not terminated **do**
- 3 Initialize a random process $\mathcal{H}$ for action exploration;
- 4 Receive initial state $\mathbf{o} = (o_1, o_2, \dots, o_N)$;
- 5 **for** $t = 1$ to T_e **do**
- 6 **for** each robot i , **do** $a_i = \bar{\mu}(o_i; \theta^a) + \mathcal{H}_i$;
- 7 Execute actions $\mathbf{a} = (a_1, a_2, \dots, a_N)$ and observe reward r and new state $\mathbf{o}'$;
- 8 Obtain the locally related robot set $G = (G_1, G_2, \dots, G_N)$ of each robot;
- 9 Calculate $\phi(\mathbf{o}) = (\varphi_1(o_k)|_{k \in \mathcal{G}_1}, \varphi_2(o_j)|_{j \in \mathcal{G}_2}, \dots, \varphi_N(o_k)|_{k \in \mathcal{G}_N})$ according to (16);
- 10 Calculate $\phi(\mathbf{a}) = (\varphi_1(a_k)|_{k \in \mathcal{G}_1}, \varphi_2(a_k)|_{k \in \mathcal{G}_2}, \dots, \varphi_N(a_k)|_{k \in \mathcal{G}_N})$ according to (17);
- 11 Store $(\mathbf{o}, \mathbf{a}, \phi(\mathbf{o}), \phi(\mathbf{a}), r, \mathbf{o}')$ in replay buffer $\mathcal{D}$;
- 12 $\mathbf{o} \leftarrow \mathbf{o}'$
- 13 **for** $I = 1$ to N **do**
- 14 Sample a random minibatch of δ samples $(\mathbf{o}^j, \mathbf{a}^j, \phi(\mathbf{o})^j, \phi(\mathbf{a})^j, r^j, \mathbf{o}'^j)$ from $\mathcal{D}$;
- 15 Update critic by minimizing the loss according to (24);
- 16 Update actor using the sampled policy gradient according to (25);
- 17 **end**
- 18 Update the target networks by soft update manner:

$$\begin{aligned} \theta'^q &\leftarrow \eta \theta^q + (1 - \eta) \theta'^q \\ \theta'^a &\leftarrow \eta \theta^a + (1 - \eta) \theta'^a \end{aligned}$$
- 19 **end**
- 20 **end**

the environment to actions with the shared policy, generating initial decisions using the pretrained network.

In the policy improvement stage, we design a deviation probability δ_{i,g_i^t} depending on two factors to assess the probability of a robot deviating from the current policy or target task. First, we consider the correlation between the number of robots already assigned to the currently selected target task $h_{g_i^t}^t$ and the highest number of robots that can be assigned to the task $\bar{h}_{g_i^t}$. If $h_{g_i^t}^t$ is closer to $\bar{h}_{g_i^t}$, there is a higher probability of deviation. This is because we encourage robots to explore tasks with fewer assigned robots

$$\bar{h}_{g_i^t} \otimes h_{g_i^t}^t = \begin{cases} \bar{h}_{g_i^t} - h_{g_i^t}^t, & \text{if } \bar{h}_{g_i^t} > h_{g_i^t}^t \\ 0, & \text{otherwise.} \end{cases} \quad (26)$$

Algorithm 2 Distributed Policy Improvements and Execution

Input : Robot i 's policy network $\bar{\mu}^*$, state s_i action space $\mathbf{a}_i$

Output: Optimized allocation strategy $\bar{\mu}_{\text{finl}}$

- 1 **for** $t = 1$ to T_e **do**
- 2 // Policy execution
- 3 **if** $c_i^t = 1$ **then**
- 4 obtain observation o_i^t of robot i ;
- 5 $g_i^t \leftarrow a_i^t = \bar{\mu}^*(o_i^t|_{\theta^a})$;
- 6 // Policy improvement
- 7 Calculate divergence probability δ_{i,g_i^t} and $\xi = \text{random}(0,1)$;
- 8 **if** $\xi < \delta_{i,g_i^t}$ **then**
- 9 $a_i^t \leftarrow g_i^t = \text{argmax}_{j \in \mathcal{M}} \varphi_1 r_{i,j} - d_{i,j}$;
- 10 **end**
- 11 $s_i^{t+1} \leftarrow \mathcal{P}_i(s_i^t, a_i^t)$;
- 12 **end**

Second, the correlation between the number of neighboring robots adopting the same strategy. Let α_i represent the number of neighbor agents that can be observed by robot i , and β_i represent the number of robots i' that adopt the same strategy as robot i , i.e., $g_{i'}^t = g_i^t, i' \in \mathcal{N}_i^t$. By calculating the difference $\alpha_i - \beta_i$, we can assess the level of potential conflict with neighboring robots. In order to avoid tasks that may conflict with neighboring robots, we encourage robots to prioritize tasks where the difference between α_i and β_i is smaller. This ensures that robot i selects a task that is less likely to cause conflicts or overlap with the tasks of its neighboring robots. Finally, based on the above discussions, we design the deviation probability as $\delta_{i,g_i^t} = e^{-\frac{(\bar{h}_{g_i^t} \otimes h_{g_i^t}^t)(\alpha_i - \beta_i)}{h_{g_i^t}^t}}$.

Remark: Although our focus is primarily on algorithmic efficiency and scalability, interpretability is also critical for safe and reliable multirobot cooperation. In physical robotics domains, understanding how local observations, reward signals, and cooperative behaviors emerge in MARL systems can help diagnose unexpected outcomes and ensure adherence to physical constraints. To illustrate LIA-MADDPG's interpretability, we employ a high-fidelity physics simulator and visualization techniques in our experiments, clarifying how LIA shapes collective strategies. Additional details on interpretability methods and evaluations are provided in Supplementary Material B.

IV. EXPERIMENTS

In this section, we will evaluate LIA-MADDPG in a large-scale multirobot system to assess its efficiency and scalability. Experiments were run on an Intel Core i7 CPU (2.8 GHz) with a single Nvidia 3090 GPU.

A. Experimental Setup

1) *Simulation Environment:* We simulate a 1000*1000 m² obstacle-free 2-D space. Each training episode starts with N robots and M tasks randomly positioned (coordinates scaled to $[0, 1]$). Robot velocities are sampled from $[2, 5]$ m/s, while

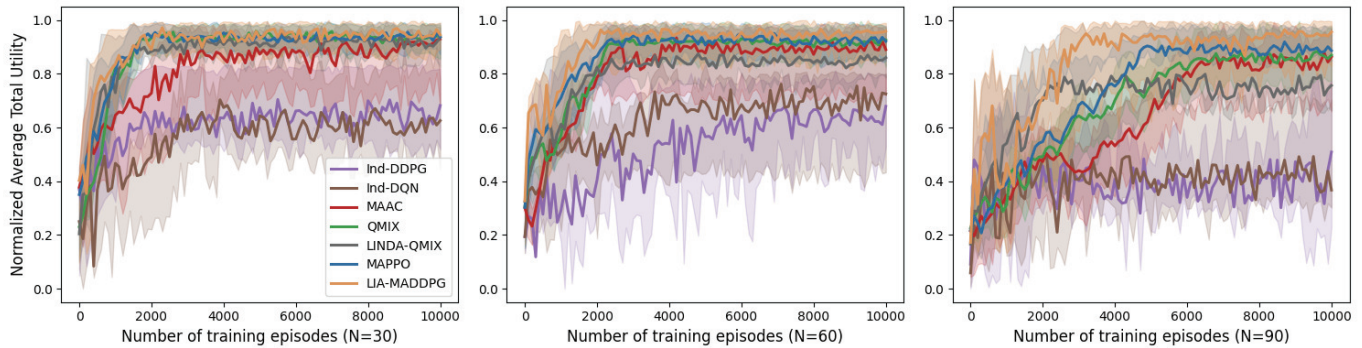


Fig. 2. Normalized average episode utility by LIA-MADDPG and six comparison RL-based methods in different robot scales. The solid and dashed lines show the mean and standard deviation of the results over ten runs, respectively.

tasks move at $[0.5, 1]$ m/s with random turning angles in $[-\pi, \pi]$. The binding distance between a robot and task is $d_{\text{bind}} = 30$ m; the maximum demand for each task $\bar{h}_j$ is set to $[N/M]$. Each robot can detect up to ten neighbors.

2) *Training Process Configuration*: Each episode lasts up to 150 time steps. Reward parameters are $\varphi_1 = 10$, $\varphi_2 = -0.001$, and $\varphi_3 = 1$. We use two-layer MLPs for both the shared policy network $\bar{G}$ and value network $\bar{\mu}$, each with 128 neurons per layer. The Adam optimizer is applied with learning rates of 0.001, 0.002. We use a replay buffer of size 5000, a batch size of 64, and a target network update rate of $\eta = 0.01$.

3) *Performance Metrics*: Performance comparison among different algorithms is based on three key metrics: normalized average total utility (NATU), which evaluates the effectiveness of an algorithm in task allocation by assessing overall rewards; normalized average time cost, measuring task completion efficiency within a set timeframe; and dominance rate (DR), indicating the frequency at which one algorithm outperforms others in terms of total rewards over a series of sample processes.

B. Training Results and Comparisons

We compare LIA-MADDPG against six reinforcement-learning baselines: Ind-DQN [44], Ind-DDPG [42], MAAC [28], QMIX [29], LINDA-QMIX [45], and MAPPO [30]. These algorithms share the same network architecture and hyperparameters for fairness. We evaluate performance with $N \in \{30, 60, 90\}$ robots.

The convergence curves depicted in Fig. 2 show that LIA-MADDPG consistently achieves near-optimal normalized average episode rewards across all tested robot counts, highlighting its robustness in managing the increasing complexity of state and action spaces. Though other MARL methods reach comparable performance at $N = 30$, they suffer from slower convergence, greater reward variability, and lower average rewards overall. In particular, LINDA-QMIX, despite quickly attaining 78% of the optimal normalized reward at $N = 90$, shows no further reward improvement during training, with significant fluctuations in its convergence curve. This is due to the large number of robots and the relatively homogenous state information in our scenario, which hampers LINDA's

ability to effectively decompose local information and develop individual teammate awareness. On the other hand, Ind-DQN and Ind-DDPG, which ignore inter-agent interactions, deliver the poorest performance, underscoring the crucial role of such dynamics in ensuring effective robot coordination. Our findings reaffirm LIA-MADDPG's superior convergence speed, stability, and overall performance, especially in large-scale environments.

1) *Comparison With Existing Methods*: Following the training phase, we carry out an extensive evaluation of our policy's performance through 100 distinct initial test scenarios, characterized by randomly assigned task locations and robot departure points. These tests span three different robot population sizes ($N = 30, 60$, and 90). As detailed in Table I, our method consistently outperforms traditional MARL methods, especially in the NATU metric. Though the other methods are competitive at a smaller scale ($N = 30$), our approach demonstrates significant superiority at larger scales, achieving NATU values up to twice as high as those of individualistic algorithms like Ind-DQN and Ind-DDPG at $N = 90$.

Regarding the normalized average task completion time (NATC), our method demonstrates the lowest time consumption across all scales, underscoring its superior efficiency compared to both single-agent and other multiagent strategies. This emphasizes the robustness and consistent performance of our algorithm. In terms of the DR, our method slightly outperforms other MARL algorithms at smaller scales and shows a significant advantage as the scale increases. At $N = 90$, our method achieves a DR of nearly 80%, demonstrating its effectiveness and scalability. In stark contrast, methods like MAPPO experience a notable decline in performance as N increases, with DR values dropping significantly—MAPPO's DR falls to just 10% at $N = 90$. LINDA-enhanced QMIX (LINDA-QMIX) faces similar issues, and in fact, the problem is even more pronounced. This further supports the idea that the LINDA module is particularly useful in small-scale heterogeneous environments like StarCraft. However, in our problem setting, agents only need to focus on nearby relevant agents when selecting target tasks, making global state awareness unnecessary. The experimental results demonstrate that our method not only maintains high efficiency in task com-

TABLE I

STATISTICAL COMPARISON OF THE TRAINED POLICIES WITH LIA-MADDPG (OURS), IND-DQN, IND-DDPG, MAAC, QMIX, LINDA-QMIX, AND MAPPO

Metrics	<i>N</i>	Ind-DDPG	Ind-DQN	MAAC	QMIX	LINDA-QMIX	MAPPO	LIA-MADDPG
<i>NATU</i>	30	0.458±0.153	0.454±0.185	0.742±0.087	0.769±0.080	0.778±0.081	0.775±0.075	0.793±0.079
	60	0.414±0.145	0.405±0.187	0.770±0.072	0.806±0.067	0.801±0.064	0.828±0.057	0.864±0.059
	90	0.282±0.127	0.295±0.145	0.666±0.079	0.730±0.073	0.705±0.062	0.775±0.060	0.863±0.052
<i>NATC</i>	30	0.395±0.179	0.387±0.187	0.372±0.187	0.357±0.187	0.331±0.188	0.348±0.185	0.325±0.123
	60	0.529±0.182	0.527±0.187	0.460±0.173	0.431±0.172	0.451±0.171	0.418±0.170	0.364±0.154
	90	0.447±0.171	0.450±0.174	0.397±0.165	0.362±0.160	0.378±0.164	0.309±0.142	0.260±0.138
<i>DR</i>	30	2%	3%	12%	10%	15%	25%	33%
	60	1%	2%	6%	14%	9%	23%	45%
	90	0%	0%	2%	7%	3%	10%	78%

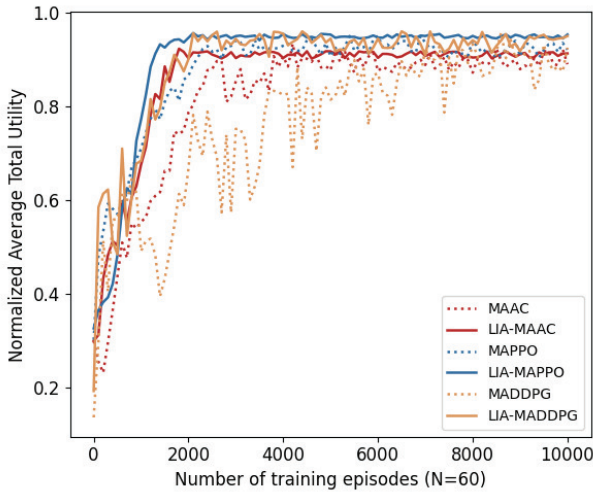


Fig. 3. Normalized average episode utility by three enhanced MARL algorithms and their traditional versions in 60 robot scales.

pletion across varying scales but also significantly improves performance metrics compared to existing algorithms, affirming its superiority across diverse operational contexts.

C. Performance Analysis of LIA

In this section, we investigate the effects of the LIA module and the policy improvement techniques as detailed in Section III.

To validate the effectiveness of the LIA module, we will design two groups of comparative experiments. The first group evaluates its applicability in the MARL framework, while the second investigates the effectiveness of its key components, including the local relevance set and the aggregation weight mechanism.

1) *LIA Applicability Evaluation*: The first set of experiments focuses on assessing the general applicability of the LIA module across different MARL algorithms. Specifically, we integrate the LIA module into three mainstream algorithms: MADDPG, MAAC, and MAPPO. Performance comparisons are made between the LIA-enhanced versions (LIA-MADDPG, LIA-MAAC, LIA-MAPPO) and their respective baselines, which do not include the LIA mod-

ule. The results, illustrated in Fig. 3, demonstrate that incorporating the LIA module significantly accelerates convergence during training. This is reflected in the rapid increase in NATU during the early stages for all LIA-enhanced algorithms. Additionally, the LIA module improves training stability, as evidenced by reduced fluctuations in the utility curves compared to the baseline versions.

To ensure the robustness of these findings, we conduct a statistical evaluation using 100 randomly generated test cases. As shown in Table II, the LIA-enhanced algorithms consistently outperform their baselines across key metrics, including NATU, NATC, and DR. However, in a few cases, the inclusion of the LIA module resulted in a slightly increased variance, suggesting a trade-off between performance improvement and result consistency. To further validate the significance of the observed improvements, we performed t-tests to analyze the differences between the LIA-enhanced and baseline algorithms at various problem scales. Table III reveals that all comparisons exhibit statistically significant differences, with p-values consistently below 0.01. These results confirm that the LIA module provides meaningful and robust performance gains, highlighting its general applicability.

2) *LIA Component Effectiveness Validation*: The second set of experiments examines the internal components of the LIA module, specifically the relevance set (local versus global) and the aggregation weight mechanism (attention-based versus distance-based). To systematically evaluate these design choices, we tested four configurations—Glob-att, Glob-dis, Local-att, and Local-dis—by integrating them into the MADDPG framework and testing under varying numbers of robots. Two new metrics are introduced to describe convergence performance: 1) convergence iterations ($\mathcal{N}_{TR}$) defined as the number of iterations required to reach the 95% confidence interval and 2) highest normalized reward ($\mathcal{R}_{NH}$), defined as the maximum reward achieved during training. The results are summarized in Table IV and Fig. 4.

a) *Global Versus Local Relevance Sets*: Configurations with the local relevance set (Local-att and Local-dis) outperformed global relevance set configurations (Glob-att and Glob-dis) across all metrics. Local relevance sets significantly reduced $\mathcal{N}_{TR}$ and achieved higher $\mathcal{R}_{NH}$, with the performance gap increasing as the environment scale grew. This demon-

TABLE II

STATISTICAL COMPARISON OF THE POLICIES LEARNED WITH THREE ENHANCED MARL ALGORITHMS (LIA-MADDPG, LIA-MAPPO, AND LIA-MAAC) AND THEIR TRADITIONAL VISIONS (MADDPG, MAPPO, AND MAAC)

Metrics	N	MAAC	LIA-MAAC	MAPPO	LIA-MAPPO	MADDPG	LIA-MADDPG
<i>NATU</i>	30	0.742 ± 0.086	0.784 ± 0.117	0.774 ± 0.075	0.803 ± 0.067	0.749 ± 0.108	0.793 ± 0.078
	60	0.770 ± 0.072	0.835 ± 0.062	0.828 ± 0.057	0.859 ± 0.063	0.782 ± 0.105	0.864 ± 0.059
	90	0.666 ± 0.079	0.755 ± 0.065	0.774 ± 0.060	0.884 ± 0.065	0.693 ± 0.141	0.863 ± 0.052
<i>NATC</i>	30	0.372 ± 0.187	0.332 ± 0.136	0.348 ± 0.185	0.303 ± 0.112	0.356 ± 0.127	0.325 ± 0.123
	60	0.460 ± 0.173	0.382 ± 0.127	0.418 ± 0.170	0.336 ± 0.112	0.464 ± 0.166	0.364 ± 0.154
	90	0.397 ± 0.165	0.296 ± 0.124	0.309 ± 0.142	0.273 ± 0.110	0.356 ± 0.161	0.260 ± 0.138
<i>DR</i>	30	42%	58%	38%	62%	35%	65%
	60	31%	69%	26%	74%	20%	80%
	90	25%	75%	12%	88%	10%	90%

TABLE III

T-TEST RESULTS FOR ALGORITHM COMPARISONS AT DIFFERENT SCALES

Algorithm Comparison	N	Mean $\pm$ Std. Dev. (Without LIA)	Mean $\pm$ Std. Dev. (With LIA)	t-statistic	p-value
MAAC vs LIA-MAAC	30	0.742 ± 0.086	0.784 ± 0.117	2.853	0.005
	60	0.770 ± 0.072	0.835 ± 0.062	6.850	0.000**
	90	0.666 ± 0.079	0.755 ± 0.065	8.534	0.000**
MAPPO vs LIA-MAPPO	30	0.774 ± 0.075	0.803 ± 0.067	2.800	0.006
	60	0.828 ± 0.057	0.859 ± 0.063	3.495	0.001
	90	0.774 ± 0.060	0.884 ± 0.065	12.552	0.000**
MADDPG vs LIA-MADDPG	30	0.749 ± 0.108	0.793 ± 0.078	3.307	0.001
	60	0.782 ± 0.105	0.864 ± 0.059	6.707	0.000**
	90	0.693 ± 0.141	0.863 ± 0.052	11.089	0.000**

TABLE IV

PERFORMANCE COMPARISON OF LIA MODULE CONFIGURATIONS UNDER DIFFERENT AGENT SCALES

Configuration	Scale 60				Scale 90			
	Glob-att	Local-att	Glob-dis	Local-dis	Glob-att	Local-att	Glob-dis	Local-dis
$\mathcal{N}_{TR} (\times 10^4)$	0.6	0.45	<u>0.3</u>	0.25	1.4	1.1	<u>0.4</u>	0.32
$\mathcal{R}_{NH}$	0.925 ± 0.04	0.947 ± 0.013	0.915 ± 0.13	<u>0.942 ± 0.016</u>	0.895 ± 0.05	<u>0.925 ± 0.04</u>	0.901 ± 0.03	0.935 ± 0.016
<i>NATU</i>	0.561 ± 0.093	0.738 ± 0.071	0.553 ± 0.086	<u>0.712 ± 0.65</u>	0.438 ± 0.125	<u>0.645 ± 0.86</u>	0.523 ± 0.04	0.689 ± 0.056
<i>NATC</i>	0.365 ± 0.124	0.360 ± 0.086	0.372 ± 0.44	<u>0.361 ± 0.65</u>	0.425 ± 0.114	<u>0.410 ± 0.54</u>	0.413 ± 0.84	0.405 ± 0.94
<i>DR</i>	12%	<u>36%</u>	15%	38%	8%	<u>39%</u>	9%	45%

strates their effectiveness in leveraging relevant neighboring information to enhance scalability and generalization.

b) Attention-Based Versus Distance-Based Weighting:

Glob-att and Local-att achieved better results in medium-scale environments (e.g., 60 robots) by flexibly allocating weights based on importance, leading to higher rewards and utility stability. However, the additional computational cost of calculating attention weights increases $\mathcal{N}_{TR}$ in larger environments. Glob-dis and Local-dis demonstrate superior efficiency in larger-scale scenarios (e.g., 90 robots) due to their simplicity. Local-dis, which balances computational cost and accuracy, can achieve faster convergence, higher rewards, and stronger DR.

c) Best Overall Configuration: Local-dis emerges as the most effective configuration, excelling in both con-

vergence speed and performance metrics, particularly in large-scale settings. By combining the efficiency of distance-based weighting with the adaptability of the local relevance set, Local-dis provides a robust and scalable solution for MARL.

Subsequently, to verify the effectiveness of the policy improvement method during the distributed execution phase, we conduct a comparative experiment. We compare the performance of a pretraining policy with LIA-MADDPG, named P_{original} , against the same policy after incorporating the policy improvement method, named P_{improved} . This comparison is implemented through testing both algorithms across 100 randomly generated scenarios, with the outcomes documented in Table V. The empirical evidence shows that P_{improved} attains higher normalized average task utility and reduced NATC

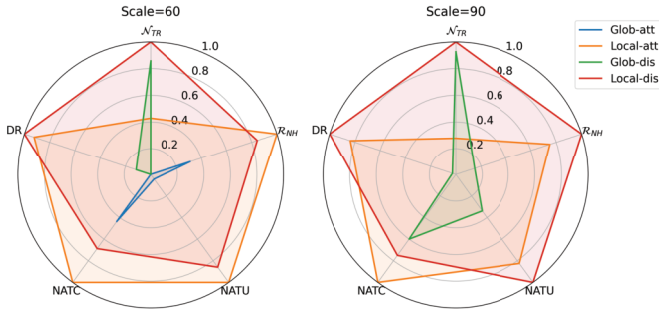


Fig. 4. Comparison of multiindicator radar charts with different configurations.

TABLE V

COMPARISON BETWEEN LIA-MADDPG AND LIA-MADDPG WITHOUT POLICY IMPROVEMENT

Metrics	N	P_{original}	P_{improved}
<i>NATU</i>	30	0.513 ± 0.207	0.538 ± 0.198
	60	0.484 ± 0.064	0.534 ± 0.219
	90	0.443 ± 0.045	0.519 ± 0.226
<i>NATC</i>	30	0.314 ± 0.168	0.293 ± 0.136
	60	0.4591 ± 0.154	0.364 ± 0.146
	90	0.422 ± 0.199	0.306 ± 0.189
<i>DR</i>	30	31%	69%
	60	21%	79%
	90	8%	92%

compared to P_{original} . This difference highlights the significant role of the policy improvement method in encouraging robots to actively explore and adjust their strategies to optimize utility. Additionally, the task completion rate (DR), detailed in the third column of Table V, indicates that the version lacking the strategy improvement method performs adequately only in smaller settings ($N = 30$). However, as the number of robots increases, its performance notably deteriorates, underscoring the policy improvement method's essential role in boosting algorithm performance under more complex and challenging conditions. These findings robustly demonstrate the critical importance of the policy improvement method, particularly in effectively scaling up the system to handle larger scenarios. This underlines that integrating such methods is crucial for enhancing the adaptability and efficiency of the system in dynamic and unpredictable environments.

D. Scalability Performance of LIA-MADDPG

In order to rigorously test the scalability of our LIA-MADDPG method across various sizes of robotic systems, we employ a structured experimental approach. We initially train our model on a medium-scale system with $N = 60$ robots and $M = 10$ tasks. Subsequently, we test this trained model on three different scales: a small-scale system ($N = 30$, $M = 5$), a medium-scale system ($N = 100$, $M = 8$), and a large-scale system ($N = 300$, $M = 10$). Each of these scales is subjected to 100 varied initial test scenarios to ensure comprehensive comparative analysis.

TABLE VI

COMPARISON BETWEEN LIA-MADDPG AND THE GREEDY CHOICE IN DIFFERENT SCALE SCENARIOS

Metrics	Scale	Greedy choice	LIA-MADDPG
<i>NATU</i>	Small	0.498 ± 0.207	0.525 ± 0.198
	Medium	0.420 ± 0.064	0.534 ± 0.219
	Large	0.310 ± 0.045	0.509 ± 0.226
<i>NATC</i>	Small	0.338 ± 0.168	0.283 ± 0.136
	Medium	0.331 ± 0.154	0.238 ± 0.146
	Large	0.461 ± 0.199	0.336 ± 0.189
<i>DR</i>	Small	42%	58%
	Medium	32%	68%
	Large	18%	72%

For our baseline comparison, we selected the well-established greedy choice method [46]. This heuristic method processes decision-making at an individual robot level, aiming to maximize personal rewards based on real-time task status, without considering the interaction and cooperation among robots. In contrast, our LIA-MADDPG method integrates these interactions, which is crucial for coordinated tasks in multirobot systems. To ensure a fair evaluation, both methods are implemented with the same utility function and identical initial conditions for each test case.

The effectiveness and performance metrics, specifically the task completion rate (DR), normalized average task utility, and NATC, are carefully analyzed. The results are presented in Table VI, and the DR results are displayed in a bar chart in Fig. 5.

Regarding NATU and NATC, it can be seen from Table VI that our method consistently outperforms the greedy choice method across all scale scenarios. Notably, in the small and medium-scale scenarios, our method exhibits nearly a 15% improvement in NATU and a 20% reduction in NATC compared to the greedy choice method. Even more, in the large-scale scenario, our method showcases remarkable performance, achieving a remarkable 64.2% improvement in NATU and a substantial 36.9% reduction in NATC when compared to the greedy choice method. These results underscore the exceptional scalability of our proposed method, particularly in large-scale scenarios. From the perspective of DR, our method outperforms the greedy choice method in 51 out of 100 training sets in the small-scale scenario. As the number of robots increases, our method's DR gradually expands over the greedy choice method. In large-scale problems, our method even achieves an impressive DR as high as 71%, further emphasizing the substantial advantages of our approach in addressing large-scale problems.

For completeness, we conducted additional experiments using a high-fidelity PyBullet simulation that precisely models robot dynamics and task motions in real time. These experiments validate our algorithm's reliability under realistic physical constraints and its ability to transition smoothly from simulation to real-world deployment. Additionally, we compare our method against a baseline greedy approach, demonstrating the advantages of LIA-MADDPG in terms of path efficiency, convergence speed, and collaborative

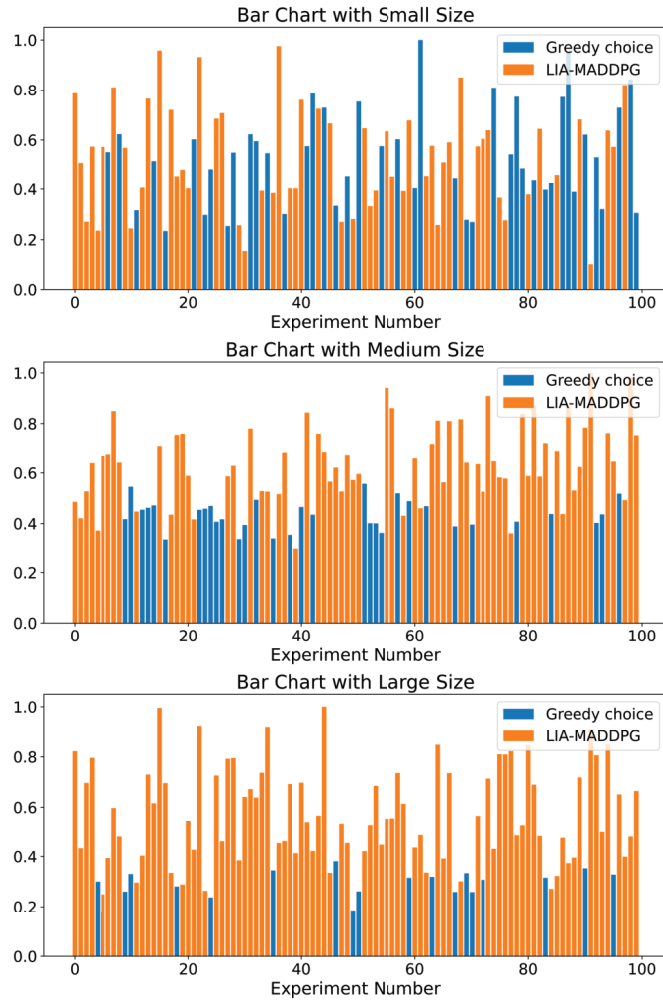


Fig. 5. Bar plot shows the DR statistics comparing our method (LIA-MADDPG) with the greedy choice method across different scales (small, medium, large). Each subfigure reflects performance over 100 experiments, where the bar color indicates the better-performing algorithm.

synergy. Due to space limitations, detailed explanations, figures, and quantitative results for this physics-based experiment are provided in Appendix C of the supplementary material.

V. CONCLUDING REMARKS

This research tackled the robot swarm task allocation problem in highly dynamic environments by framing it as a Dec-POMDP and introducing LIA-MADDPG, a multiagent deep reinforcement learning method. During centralized training, robots employ an LIA module to focus on relevant observations, while in the distributed execution phase, policy-improvement strategies enhance allocation solutions. Extensive experiments confirm the approach's superior convergence speed and cooperative efficacy, particularly in large-scale, homogeneous settings.

Nonetheless, the current design is not directly tailored for heterogeneous agent scenarios, where robots may differ significantly in their capabilities, tasks, or information requirements. Another important consideration missing so far is collision

avoidance, which becomes critical as robot density and environmental complexity grow. A promising future direction is to extend LIA-MADDPG to handle both heterogeneity and collision dynamics, thereby achieving robust, safe, and efficient performance across a wider range of real-world multirobot applications.

REFERENCES

- [1] X. Dong, Y. Li, C. Lu, G. Hu, Q. Li, and Z. Ren, "Time-varying formation tracking for UAV swarm systems with switching directed topologies," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 12, pp. 3674–3685, Dec. 2019.
- [2] O. T. Abdulhae, J. S. Mandeep, and M. Islam, "Cluster-based routing protocols for flying ad hoc networks (FANETs)," *IEEE Access*, vol. 10, pp. 32981–33004, 2022.
- [3] G. Chen and Y.-D. Song, "Cooperative tracking control of nonlinear multiagent systems using self-structuring neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 25, no. 8, pp. 1496–1507, Aug. 2014.
- [4] M. Campion, P. Ranganathan, and S. Faruque, "UAV swarm communication and control architectures: A review," *J. Unmanned Vehicle Syst.*, vol. 7, no. 2, pp. 93–106, Jun. 2019.
- [5] X. Wang, M. Peng, H. Lin, Y. Wu, and X. Fan, "A privacy-enhanced multiarea task allocation strategy for healthcare 4.0," *IEEE Trans. Ind. Inform.*, vol. 19, no. 3, pp. 2740–2748, Mar. 2023.
- [6] T. Chen, Y. Cai, L. Chen, and X. Xu, "Trajectory and velocity planning method of emergency rescue vehicle based on segmented three-dimensional quartic Bezier curve," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 3, pp. 3461–3475, Mar. 2023.
- [7] H. Gao, J. Feng, Y. Xiao, B. Zhang, and W. Wang, "A UAV-assisted multi-task allocation method for mobile crowd sensing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 3790–3804, Jul. 2022.
- [8] K. Liu and Y. Zhang, "Distributed dynamic task allocation for moving target tracking of networked mobile robots using k -WTA network," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 3, pp. 5795–5802, Mar. 2024.
- [9] Q. Zhang, L. Gui, F. Hou, J. Chen, S. Zhu, and F. Tian, "Dynamic task offloading and resource allocation for mobile-edge computing in dense cloud RAN," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 3282–3299, Apr. 2020.
- [10] Q. Peng, H. Wu, and R. Xue, "Review of dynamic task allocation methods for UAV swarms oriented to ground targets," *Complex Syst. Model. Simul.*, vol. 1, no. 3, pp. 163–175, Sep. 2021.
- [11] A. Khamis, A. Hussein, and A. Elmogy, "Multi-robot task allocation: A review of the state-of-the-art," in *Cooperative Robots and Sensor Networks*, vol. 554. Cham, Switzerland: Springer, 2015, pp. 31–51.
- [12] L. Wang, J. Liu, J. Hu, Q. Zhuge, and E. H.-M. Sha, "Optimal assignment for tree-structure task graph on heterogeneous multicore systems considering time constraint," in *Proc. IEEE 6th Int. Symp. Embedded Multicore SoCs*, Sep. 2012, pp. 121–127.
- [13] S. An and H. J. Kim, "Simultaneous task assignment and path planning using mixed-integer linear programming and potential field method," in *Proc. 13th Int. Conf. Control, Autom. Syst. (ICCAS)*, Oct. 2013, pp. 1845–1848.
- [14] T. Issac, S. Silas, and E. B. Rajasingh, "Investigations on PSO based task assignment algorithms for heterogeneous wireless sensor network," in *Proc. 2nd Int. Conf. Signal Process. Commun. (ICSPC)*, Mar. 2019, pp. 89–93.
- [15] W. C. Stirling and M. A. Goodrich, "Conditional preferences for social systems," in *Proc. IEEE Int. Conf. Syst., Man Cybern. E-Syst. E-Man Cybern. Cyberspace*, vol. 2, Sep. 2001, pp. 995–1000.
- [16] V. G. Lopez, F. L. Lewis, Y. Wan, E. N. Sanchez, and L. Fan, "Solutions for multiagent pursuit-evasion games on communication graphs: Finite-time capture and asymptotic behaviors," *IEEE Trans. Autom. Control*, vol. 65, no. 5, pp. 1911–1923, May 2020.
- [17] J. C. Amorim, V. Alves, and E. P. de Freitas, "Assessing a swarm-GAP based solution for the task allocation problem in dynamic scenarios," *Expert Syst. Appl.*, vol. 152, Aug. 2020, Art. no. 113437.
- [18] M. M. Zavlanos and G. J. Pappas, "Dynamic assignment in distributed motion planning with local coordination," *IEEE Trans. Robot.*, vol. 24, no. 1, pp. 232–242, Feb. 2008.
- [19] D. Panagou, M. Turpin, and V. Kumar, "Decentralized goal assignment and safe trajectory generation in multirobot networks via multiple Lyapunov functions," *IEEE Trans. Autom. Control*, vol. 65, no. 8, pp. 3365–3380, Aug. 2020.

- [20] J. Yu, S.-J. Chung, and P. G. Voulgaris, "Target assignment in robotic networks: Distance optimality guarantees and hierarchical strategies," *IEEE Trans. Autom. Control*, vol. 60, no. 2, pp. 327–341, Feb. 2015.
- [21] D. P. Bertsekas, "A new algorithm for the assignment problem," *Math. Program.*, vol. 21, no. 1, pp. 152–171, Dec. 1981.
- [22] R. G. Smith, "The contract net protocol: High-level communication and control in a distributed problem solver," *IEEE Trans. Comput.*, vol. C-29, no. 12, pp. 1104–1113, Dec. 1980.
- [23] L. Luo, N. Chakraborty, and K. Sycara, "Distributed algorithms for multirobot task assignment with task deadline constraints," *IEEE Trans. Autom. Sci. Eng.*, vol. 12, no. 3, pp. 876–888, Jul. 2015.
- [24] D.-H. Lee, S. A. Zaheer, and J.-H. Kim, "A resource-oriented, decentralized auction algorithm for multirobot task allocation," *IEEE Trans. Autom. Sci. Eng.*, vol. 12, no. 4, pp. 1469–1481, Oct. 2015.
- [25] J. Zhang, G. Wang, and Y. Song, "Task assignment of the improved contract net protocol under a multi-agent system," *Algorithms*, vol. 12, no. 4, p. 70, Apr. 2019.
- [26] I. Jang, H.-S. Shin, and A. Tsourdos, "Anonymous hedonic game for task allocation in a large-scale multiple agent system," *IEEE Trans. Robot.*, vol. 34, no. 6, pp. 1534–1548, Dec. 2018.
- [27] S. A. Akgun, M. Ghafurian, M. Crowley, and K. Dautenhahn, "Using affect as a communication modality to improve human–robot communication in robot-assisted search and rescue scenarios," *IEEE Trans. Affect. Comput.*, vol. 14, no. 4, pp. 3013–3030, Oct. 2023.
- [28] S. Iqbal and F. Sha, "Actor-attention-critic for multi-agent reinforcement learning," in *Proc. 36th Int. Conf. Mach. Learn.* K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97, Jun. 2019, pp. 2961–2970.
- [29] Q. Chen, "NQMIx: Non-monotonic value function factorization for deep multi-agent reinforcement learning," 2021, *arXiv:2104.01939*.
- [30] C. Yu et al., "The surprising effectiveness of PPO in cooperative multi-agent games," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, Nov. 2022, pp. 24611–24624.
- [31] T. Gabel and M. Riedmiller, "Adaptive reactive job-shop scheduling with reinforcement learning agents," *Int. J. Inf. Technol. Intell. Comput.*, vol. 24, no. 4, pp. 14–18, 2008.
- [32] S. Qu, J. Wang, and J. Jasperneite, "Dynamic scheduling in modern processing systems using expert-guided distributed reinforcement learning," in *Proc. 24th IEEE Int. Conf. Emerg. Technol. Factory Autom. (ETFA)*, Sep. 2019, pp. 459–466.
- [33] C.-L. Liu, C.-C. Chang, and C.-J. Tseng, "Actor-critic deep reinforcement learning for solving job shop scheduling problems," *IEEE Access*, vol. 8, pp. 71752–71762, 2020.
- [34] S. Luo, L. Zhang, and Y. Fan, "Real-time scheduling for dynamic partial-no-wait multiobjective flexible job shop by deep reinforcement learning," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 4, pp. 3020–3038, Oct. 2022.
- [35] X. Wang, L. Zhang, T. Lin, C. Zhao, K. Wang, and Z. Chen, "Solving job scheduling problems in a resource preemption environment with multi-agent reinforcement learning," *Robot. Comput.-Integr. Manuf.*, vol. 77, Oct. 2022, Art. no. 102324.
- [36] F. A. Oliehoek and C. Amato, *A Concise Introduction to Decentralized POMDPs*, vol. 1. Cham, Switzerland: Springer, 2016.
- [37] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30. Long Beach, CA, USA: Curran Associates, 2017, pp. 6382–6393.
- [38] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [39] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 448–456.
- [40] Y. Yang, R. Luo, M. Li, M. Zhou, W. Zhang, and J. Wang, "Mean field multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5571–5580.
- [41] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, Jun. 2017, pp. 5998–6008.
- [42] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.
- [43] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," 2015, *arXiv:1511.05952*.
- [44] V. Mnih et al., "Playing Atari with deep reinforcement learning," 2013, *arXiv:1312.5602*.
- [45] J. Cao et al., "LINDA: Multi-agent local information decomposition for awareness of teammates," *Sci. China Inf. Sci.*, vol. 66, no. 8, Aug. 2023, Art. no. 182101.
- [46] A. Vince, "A framework for the greedy algorithm," *Discrete Appl. Math.*, vol. 121, nos. 1–3, pp. 247–260, Sep. 2002.



Yang Lv received the M.S. degree in control theory and process from Kunming University of Science and Technology, Kunming, China, in 2022. He is currently pursuing the Ph.D. degree in intelligent science and technology with Tongji University, Shanghai, China.

His research interests include multiagent reinforcement learning, graph neural networks, and cluster task allocation in game-theoretic scenarios.



Jinlong Lei (Member, IEEE) received the B.E. degree in automation from the University of Science and Technology of China, Hefei, China, in 2011, and the Ph.D. degree in operations research and cybernetics from the Institute of Systems Science, Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing, China, in 2016.

She was a Post-Doctoral Fellow with the Department of Industrial and Manufacturing Engineering, The Pennsylvania State University, University Park, PA, USA, from 2016 to 2019. She is currently a Professor with the Department of Control Science and Engineering, Tongji University, Shanghai, China. Her research interests include multiagent decision-making, optimization, and game-theoretic problems in networked regimes complicated by uncertainty.



Peng Yi (Member, IEEE) received the B.E. degree in automation from the University of Science and Technology of China, Hefei, China, in 2011, and the Ph.D. degree in operations research and cybernetics from the Academy of Mathematics and Systems Science, Beijing, China, in 2016.

He was a Post-Doctoral Fellow at the University of Toronto, Toronto, ON, Canada, from 2016 to 2017; and a Post-Doctoral Associate at Washington University in St. Louis, St. Louis, MO, USA, from 2017 to 2019. He is currently a Professor at the Department of Control Science and Engineering, Tongji University, Shanghai, China. His research interest covers distributed optimization and game, neural systems, and autonomous agents.